

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..35

FUTURE CALLS

 Copy as Markdown

Future calls

A future call is a method that Serverpod runs later. You define it on a class, generate the type-safe code, and schedule it from anywhere you have access to the server. This page covers the one-off cases: run once after a delay, or once at a specific time. For repeating schedules, see [Recurring tasks](#).

Define a future call

Extend the `FutureCall` class and add the methods you want to invoke later.

```
import 'package:serverpod/serverpod.dart';

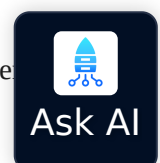
class ExampleFutureCall extends FutureCall {
  Future<void> doWork(Session session) async {
    // Do something interesting in the future here.
  }

  Future<void> doOtherWork(Session session, String data) async {
    // Do something interesting in the future here.
  }
}
```

Each method must:

- Return `Future<void>`.
- Take a `Session` as its first parameter.
- Use only [serializable types](#) for any other parameters. Typed `List`, `Map`, `Set`, and Dart records are allowed, but streams are not.

Generate the code for your future calls. With `serverpod start` running, saving the file regenerates the code. Outside a session, run `serverpod generate`.



This creates a type-safe interface in your server's `lib/src/generated/future_calls.dart` file, reachable from the `Serverpod` object as `pod.futureCalls`. You do not register these calls yourself. Serverpod does it for you when the server starts, so your `server.dart` needs no future-call setup:

```
import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  await pod.start();
}
```

NOTE

The `pod.futureCalls` accessor exists only once you have defined at least one future call and generated the code.

WARNING

Scheduling a call before the server has started throws an exception, so schedule from an endpoint or after `pod.start()` rather than while building the server.

Schedule a call

Schedule a call by chaining the timing method, the accessor for your class, and the method to invoke. The accessor is your class name with the first letter lowercased and a trailing `FutureCall` removed, so `ExampleFutureCall` becomes `example`.

Run a call one hour from now with `callWithDelay`:

```
await pod.futureCalls
  .callWithDelay(const Duration(hours: 1))
  .example
  .doWork();
```

Run a call at a specific time with `callAtTime`:

```
await pod.futureCalls
  .callAtTime(DateTime(2030, 1, 1))
  .example
  .doOtherWork('1');
```

The call runs at the exact instant you pass. A `DateTime` denotes an absolute moment, so `DateTime(2030, 1, 1)` schedules the call for midnight in the server's local time. Pass a UTC value, such as `DateTime.utc(2030, 1, 1)`, if you want the numbers to be read as UTC.

Schedule from an endpoint

Inside an endpoint you reach the server through `session.serverpod`, so you can schedule a call in response to a request:

```
class OrderEndpoint extends Endpoint {
  Future<void> placeOrder(Session session, Order order) async {
    // ... store the order ...

    // Send a follow-up an hour from now.
    await session.serverpod.futureCalls
      .callWithDelay(const Duration(hours: 1))
      .example
      .doWork();
  }
}
```

Cancel scheduled calls

Give a call an `identifier` when you schedule it so you can cancel it later. The same identifier can be applied to several calls.

```
await pod.futureCalls
  .callWithDelay(
    const Duration(hours: 1),
    identifier: 'an-identifying-string',
  )
  .example
  .doWork();
```

Cancel every not-yet-run call that shares an identifier:

```
await pod.futureCalls.cancel('an-identifying-string');
```

Calls that share an identifier run independently. The identifier only groups them so you can cancel them together, and cancelling affects only calls that have not run yet.

Inspect scheduled calls

Scheduled calls are stored in the `serverpod_future_call` database table. You can query it to see what is pending, which helps when debugging why a call did or did not run.

Execution timeout

Future calls have no execution timeout: a call runs until its method returns. If a call needs a time limit, enforce it inside the method yourself, for example with `Future.timeout` around the work.

Related

- [Recurring tasks](#): run a call on a repeating schedule.
- [Configuration](#): concurrency, scan interval, and broken-call handling.
- [Sessions](#): the session a future call receives when it runs.